

DATABASES

RELATIONAL MODEL

Relational Algebra

Konstantinos Katsifis

After completing this chapter, you should be able to

- ✚ enumerate and explain the operations of relational algebra
 - (there is a core of 5 relational algebra operators),
- ✚ write relational algebra queries of the type join–select–project,
- ✚ discuss correctness and equivalence of given relational algebra queries.

Overview

1. Introduction; Selection, Projection
2. Cartesian Product, Join
3. Set Operations
4. Outer Join

Example Database

STUDENTS

<u>SID</u>	FIRST	LAST	EMAIL
101	Ann	Smith	...
102	Michael	Jones	(null)
103	Richard	Turner	...
104	Maria	Brown	...

EXERCISES

<u>CAT</u>	<u>ENO</u>	TOPIC	MAXPT
H	1	Rel.Alg.	10
H	2	SQL	10
M	1	SQL	14

RESULTS

<u>SID</u>	<u>CAT</u>	<u>ENO</u>	POINTS
101	H	1	10
101	H	2	8
101	M	1	12
102	H	1	9
102	H	2	9
102	M	1	10
103	H	1	5
103	M	1	7

Relational Algebra (1)

- Relational algebra (RA) is a query language for the relational model with a solid theoretical foundation.
- Relational algebra is *not* visible at the user interface level (not in any commercial RDBMS, at least).
- However, almost any RDBMS uses RA to represent queries internally (for query optimization and execution).
- Knowledge of relational algebra will help in better understanding SQL and relational database systems in general.

Relational Algebra (2)

- One particular operation of relational algebra is selection.
Many operations of the relational algebra are denoted as greek letters. Selection is σ (sigma).
- For example, the operation $\sigma_{SID=101}$ selects all tuples in the input relation which have the value 101 in column SID.

Relational algebra: selection

$\sigma_{SID=101}$

RESULTS			
SID	CAT	ENO	POINTS
101	H	1	10
101	H	2	8
101	M	1	12
102	H	1	9
102	H	2	9
102	M	1	10
103	H	1	5
103	M	1	7

=

SID	CAT	ENO	POINTS
101	H	1	10
101	H	2	8
101	M	1	12

Relational Algebra (3)

- Since the output of any RA operation is some relation R again, R may be the input for another RA operation.

The operations of RA nest to arbitrary depth such that complex queries can be evaluated. The final results will always be a relation.

- A query is a term (or expression) in this relational algebra.

A query

$$\pi_{\text{FIRST, LAST}}(\text{STUDENTS} \bowtie \sigma_{\text{CAT} = 'M'}(\text{RESULTS}))$$

Relational Algebra (4)

- There are some difference between the two query languages RA and SQL:
 - ⇒ Null values are usually excluded in the definition of relational algebra, except when operations like outer join are defined.
 - ⇒ Relational algebra treats relations as sets, i.e., duplicate tuples will never occur in the input/output relations of an RA operator.

Remember: In SQL, relations are multisets (bags) and may contain duplicates. Duplicate elimination is explicit in SQL (SELECT DISTINCT).

Selection (1)

Selection

The selection σ_{φ} selects a subset of the tuples of a relation, namely those which satisfy predicate φ . Selections acts like a filter on a set.

Selection

$$\sigma_{A=1} \left(\begin{array}{c|c} A & B \\ \hline 1 & 3 \\ 1 & 4 \\ 2 & 5 \end{array} \right) = \begin{array}{c|c} A & B \\ \hline 1 & 3 \\ 1 & 4 \end{array}$$

Selection (2)

- A simple selection predicate φ has the form
$$(Term) (ComparisonOperator) (Term).$$
- $(Term)$ is an expression that can be evaluated to a data value for a given tuple:
 - ✓ an attribute name,
 - ✓ a constant value,
 - ✓ an expression built from attributes, constants, and data type operations like $+$, $-$, $*$, $/$.

Selection (3)

- (*ComparisonOperator*) is
 - ⇒ = (equals), /= (not equals),
 - ⇒ < (less than), > (greater than), ", ≥,
 - ⇒ or other data type-dependent predicates (*e.g.*, LIKE).
- Examples for simple selection predicates:
 - ⇒ LAST = 'Smith'
 - ⇒ POINTS ≥ 8
 - ⇒ POINTS = MAXPT.

Selection (4)

A few corner cases

$$\sigma_{C=1} \left(\begin{array}{c|c} A & B \\ \hline 1 & 3 \\ 1 & 4 \\ 2 & 5 \end{array} \right) = \text{⚡} \quad (\text{schema error})$$

$$\sigma_{A=A} \left(\begin{array}{c|c} A & B \\ \hline 1 & 3 \\ 1 & 4 \\ 2 & 5 \end{array} \right) = \begin{array}{c|c} A & B \\ \hline 1 & 3 \\ 1 & 4 \\ 2 & 5 \end{array}$$

$$\sigma_{1=2} \left(\begin{array}{c|c} A & B \\ \hline 1 & 3 \\ 1 & 4 \\ 2 & 5 \end{array} \right) = \underline{\underline{\begin{array}{c|c} A & B \end{array}}}$$

Selection (5)

- $\sigma_{\varphi}(R)$ corresponds to the following SQL query:

```
SELECT *
```

```
FROM R
```

```
WHERE  $\varphi$ 
```

- A different relational algebra operation called projection corresponds to the SELECT clause. Source of confusion.

Selection (6)

- More complex selection predicates may be performed using the Boolean connectives:

$\varphi_1 \wedge \varphi_2$ (“and”), $\varphi_1 \vee \varphi_2$ (“or”), $\neg \varphi_1$ (“not”).

- Note: $\sigma_{\varphi_1 \wedge \varphi_2}(R) = \sigma_{\varphi_1}(\sigma_{\varphi_2}(R))$.
- The selection predicate must permit evaluation for each input tuple in isolation. A predicate may not refer to other tuples.

Projection (1)

Projection

The **projection** π_L eliminates all attributes (columns) of the input relation but those mentioned in the **projection list** L .

Projection

$$\pi_{A,C} \left(\begin{array}{|c|c|c|} \hline A & B & C \\ \hline 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \\ \hline \end{array} \right) = \begin{array}{|c|c|} \hline A & C \\ \hline 1 & 7 \\ 2 & 8 \\ 3 & 9 \\ \hline \end{array}$$

Projection (2)

- The projection $\pi_{A_1, \dots, A_k} (R)$ produces for each input tuple $(A_1 : d_1, \dots, A_n : d_n)$ an output tuple $(A_{i_1} : d_{i_1}, \dots, A_{i_k} : d_{i_k})$.
- π may be used to reorder columns.

“ σ discards rows, π discards columns.”
- DB slang: “All attributes not in L are projected away.”

Projection (3)

- In general, the cardinalities of the input and output relations are not equal.

Projection eliminates duplicates

$$\pi_B \left(\begin{array}{|c|c|} \hline A & B \\ \hline 1 & 4 \\ 2 & 5 \\ 3 & 4 \\ \hline \end{array} \right) = \begin{array}{|c|} \hline B \\ \hline 4 \\ 5 \\ \hline \end{array}$$

Projection (4)

- If RA is used to simulate SQL, the format of the projection list is often generalized:
Attribute renaming:

$$\pi_{B_1 \leftarrow A_{i_1}, \dots, B_k \leftarrow A_{i_k}}(R) \ .$$

Computations (e.g., string concatenation via || or arithmetics via +, -, ...) to derive the value in new columns,

e.g.:

$$\pi_{SID, NAME \leftarrow FIRST \ || \ ' \ ' \ || \ LAST}(STUDENTS) \ .$$

Projection (5)

- $\pi_{A_1, \dots, A_k}(R)$ corresponds to the SQL query:

```
SELECT DISTINCT  A1, . . . , Ak
FROM              R
```

- $\pi_{B_1 \leftarrow A_1, \dots, B_k \leftarrow A_k}(R)$ is equivalent to the SQL query:

```
SELECT DISTINCT  A1 [AS] B1, . . . , Ak [AS] Bk
FROM              R
```

Selection vs. Projection

Selection vs. Projection

Selection σ

A_1	A_2	A_3	A_4

Filter some rows

Projection π

A_1	A_2	A_3	A_4

Projects all rows

Combining Operations (1)

- Since the result of any relational algebra operation is a relation again, this intermediate result may be the input of a subsequent RA operation.
- Example: retrieve the exercises solved by student with ID 102:

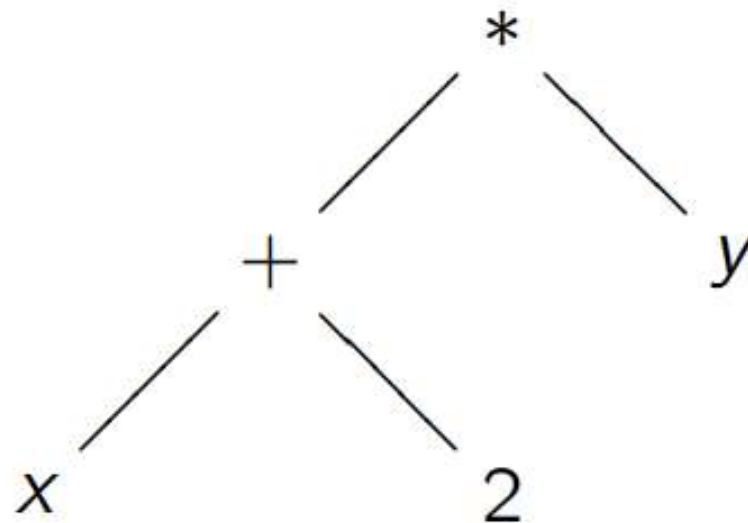
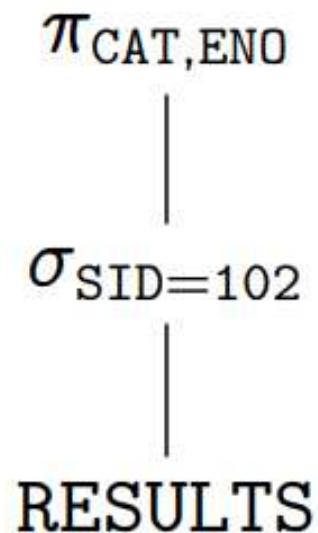
$$\pi_{\text{CAT,ENO}}(\sigma_{\text{SID}=102}(\text{RESULTS})) .$$

We can think of the intermediate result to be stored in a named temporary relation (or as a macro definition):

$$\begin{aligned} S102 &\leftarrow \sigma_{\text{SID}=102}(\text{RESULTS}); \\ \pi_{\text{CAT,ENO}}(S102) \end{aligned}$$

Combining Operations (2)

- Composite RA expressions are typically depicted as operator trees:



- In these trees, computation proceeds bottom-up. The evaluation order of sibling branches is not pre-determined.

Combining Operations (3)

- SQL permits the nesting of queries (the result of a SQL query may be used in a place of a relation name):

Nested SQL Query

```
SELECT DISTINCT CAT, ENO
FROM          (SELECT *
               FROM    RESULTS
               WHERE   SID = 102) AS S102
```

- Note that this is *not* the typical style of SQL querying.

Combining Operations (4)

- Instead, a single SQL query is equivalent to an RA operator tree containing σ , π , and (multiple) $\times$ (see below):

SELECT-FROM-WHERE Block

```
SELECT DISTINCT CAT, ENO
FROM           RESULTS
WHERE          SID = 102
```

- Really complex queries may be constructed step-by-step (using SQL's **view** mechanism), S102 may be used like a relation:

SQL View Definition

```
CREATE VIEW S102
AS SELECT *
FROM     RESULTS
WHERE    SID = 102
```


Overview

1. Introduction; Selection, Projection
2. Cartesian Product, Join
3. Set Operations
4. Outer Join

Cartesian Product (1)

- In general, queries need to combine information from several tables.
- In RA, such queries are formulated using $\times$, the Cartesian product.

Cartesian Product

The Cartesian product $R \times S$ of two relations R, S is computed by concatenating each tuple $t \in R$ with each tuple $u \in S$.

Cartesian Product (2)

Cartesian Product

<i>A</i>	<i>B</i>				<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
1	2	×		=	1	2	6	7
3	4				1	2	8	9
					3	4	6	7
					3	4	8	9

- Since attribute names must be unique within a tuple, the Cartesian product may only be applied if R, S do not share any attribute names. (This is no real restriction because we have π .)

Cartesian Product and Renaming

- $R \times S$ may be computed by the equivalent SQL query (SQL does not impose the unique column name restriction, a column A of relation R may uniquely be identified by $R.A$):

Cartesian Product in SQL

```
SELECT *  
FROM   R, S
```

- ▷ In RA, this is often formalized by means of of a **renaming operator** $\rho_X(R)$. If $sch(R) = (A_1 : D_1, \dots, A_n : D_n)$, then

$$\rho_X(R) \equiv \pi_{X.A_1 \leftarrow A_1, \dots, X.A_n \leftarrow A_n}(R) .$$

Join (1)

- The intermediate result generated by a Cartesian product may be quite large in general ($|R| = n, |S| = m \Rightarrow |R \times S| = n * m$).
- Since the combination of Cartesian product and selection in queries is common, a special operator join has been introduced.

Join

The **(theta-)join** $R \bowtie_{\theta} S$ between relations R, S is defined as

$$R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S).$$

The **join predicate** θ may refer to attribute names of R and S .

Join (2)

$Q_S(\text{STUDENTS}) \bowtie_{S.SID=R.SID} Q_R(\text{RESULTS})$

S.SID	S.FIRST	S.LAST	S.EMAIL	R.SID	R.CAT	R.ENO	R.POINTS
101	Ann	Smith	...	101	H	1	10
101	Ann	Smith	...	101	H	2	8
101	Ann	Smith	...	101	M	1	12
102	Michael	Jones	(null)	102	H	1	9
102	Michael	Jones	(null)	102	H	2	9
102	Michael	Jones	(null)	102	M	1	10
103	Richard	Turner	...	103	H	1	5
103	Richard	Turner	...	103	M	1	7

- Note: student Maria Brown does not appear in the join result.

Join (3)

- Join combines tuples from two relations **and** acts like a filter: tuples without join partner are removed.

Note: if the join is used to **follow a foreign key relationship**, then **no tuples are filtered**:

Join follows a foreign key relationship (dereference)

`RESULTS  $\bowtie_{SID=S.SID} \pi_{S.SID \leftarrow SID, FIRST, LAST, EMAIL}(STUDENTS)$`

- There are join variants which act like **filters only**: left and right **semijoin** ($\ltimes$, $\rtimes$):

$$R \ltimes_{\theta} S \equiv \pi_{sch(R)}(R \bowtie_{\theta} S) ,$$

or **do not filter** at all: **outer-join** (see below).

Natural Join

- The **natural join** provides another useful abbreviation (“RA macro”).

In the natural join $R \bowtie S$, the join predicate θ is defined to be an **conjunctive equality comparison of attributes sharing the same name** in R, S .

Natural join handles the necessary attribute renaming and projection.

Natural Join

Assume $R(A, B, C)$ and $S(B, C, D)$. Then:

$$R \bowtie S = \pi_{A,B,C,D}(\sigma_{B=B' \wedge C=C'}(R \times \pi_{B' \leftarrow B, C' \leftarrow C, D}(S)))$$

(**Note:** shared columns occur *once* in the result.)

Joins in SQL (1)

- In SQL, $R \bowtie_{\theta} S$ is normally written as

Join in SQL (“classic” and SQL-92)

SELECT	*		SELECT	*
FROM	R, S	or	FROM	$R \text{ JOIN } S \text{ ON } \theta$
WHERE	θ			

- **Note:** the left query is **exactly** the SQL equivalent of $\sigma_{\theta}(R \times S)$ we have seen before.

SQL is a **declarative language**: it is the task of the SQL optimizer to infer that this query may be evaluated using a join instead of a Cartesian product.

Algebraic Laws (1)

- A significant number **algebraic laws** hold for join which are heavily utilized by the query optimizer.
- Example: **selection push-down**.

If predicate φ **refers to attributes in S only**, then

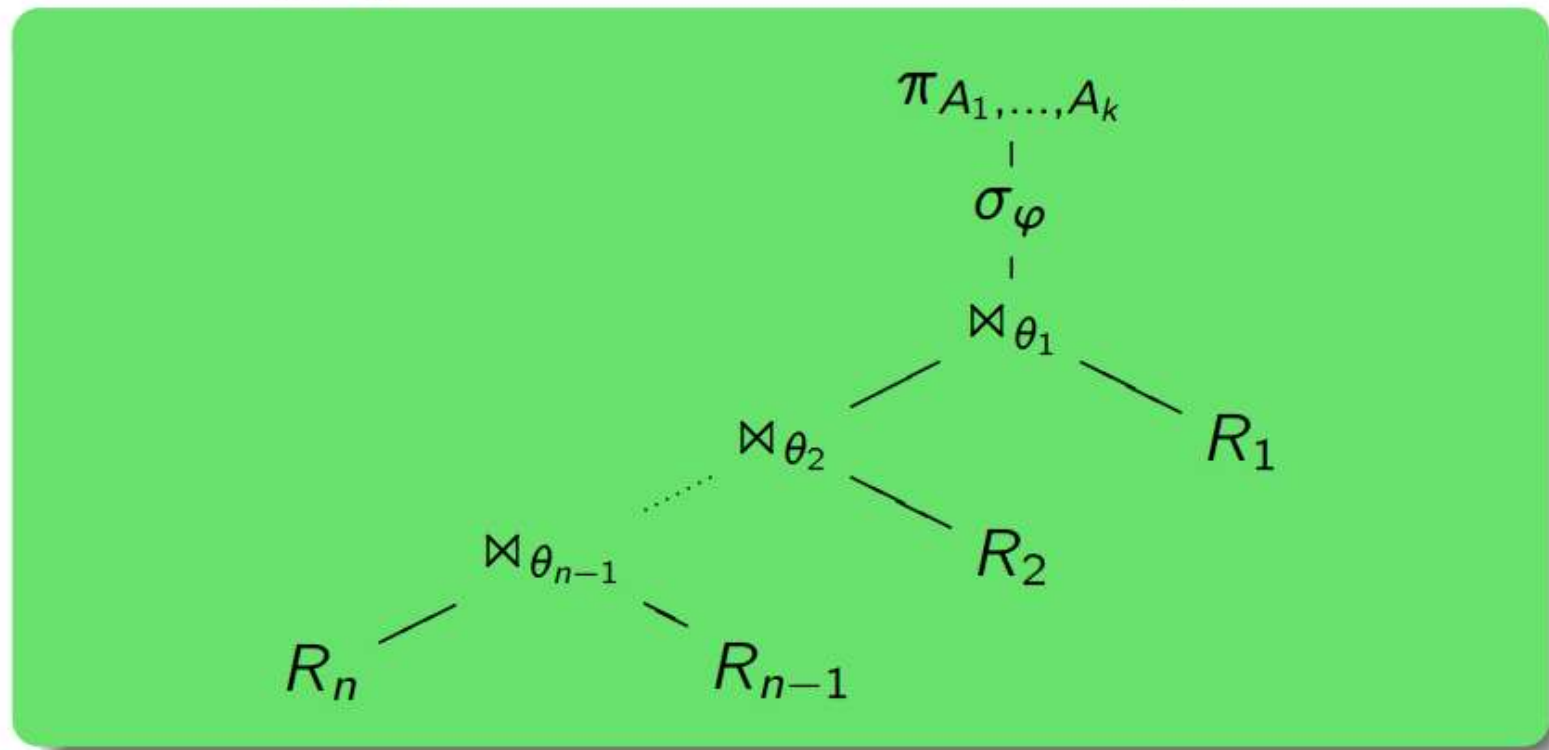
$$\sigma_{\varphi}(R \bowtie S) \equiv R \bowtie \sigma_{\varphi}(S) .$$

Selection push-down

Why is selection push-down considered one of the most significant algebraic optimizations?

A Common Query Pattern (1)

- The following operator tree structure is **very** common:



- ① **Join** all tables needed to answer the query, ② **select** the relevant tuples, ③ **project** away all irrelevant columns.

A Common Query Pattern (2)

- The select-project-join query

$$\pi_{A_1, \dots, A_k}(\sigma_{\varphi}(R_1 \bowtie_{\theta_1} R_2 \bowtie_{\theta_2} \dots \bowtie_{\theta_{n-1}} R_n))$$

has the obvious SQL equivalent

SELECT	$A_1, \dots, A_k$
DISTINCT FROM	$R_1, \dots, R_n$
WHERE	φ
AND	θ_1 AND ... AND θ_{n-1}

- It is a common source of errors to forget a join condition: think of the scenario $R(A, B)$, $S(B, C)$, $T(C, D)$ when attributes A, D are relevant for the query output.

Overview

1. Introduction; Selection, Projection
2. Cartesian Product, Join
3. Set Operations
4. Outer Join

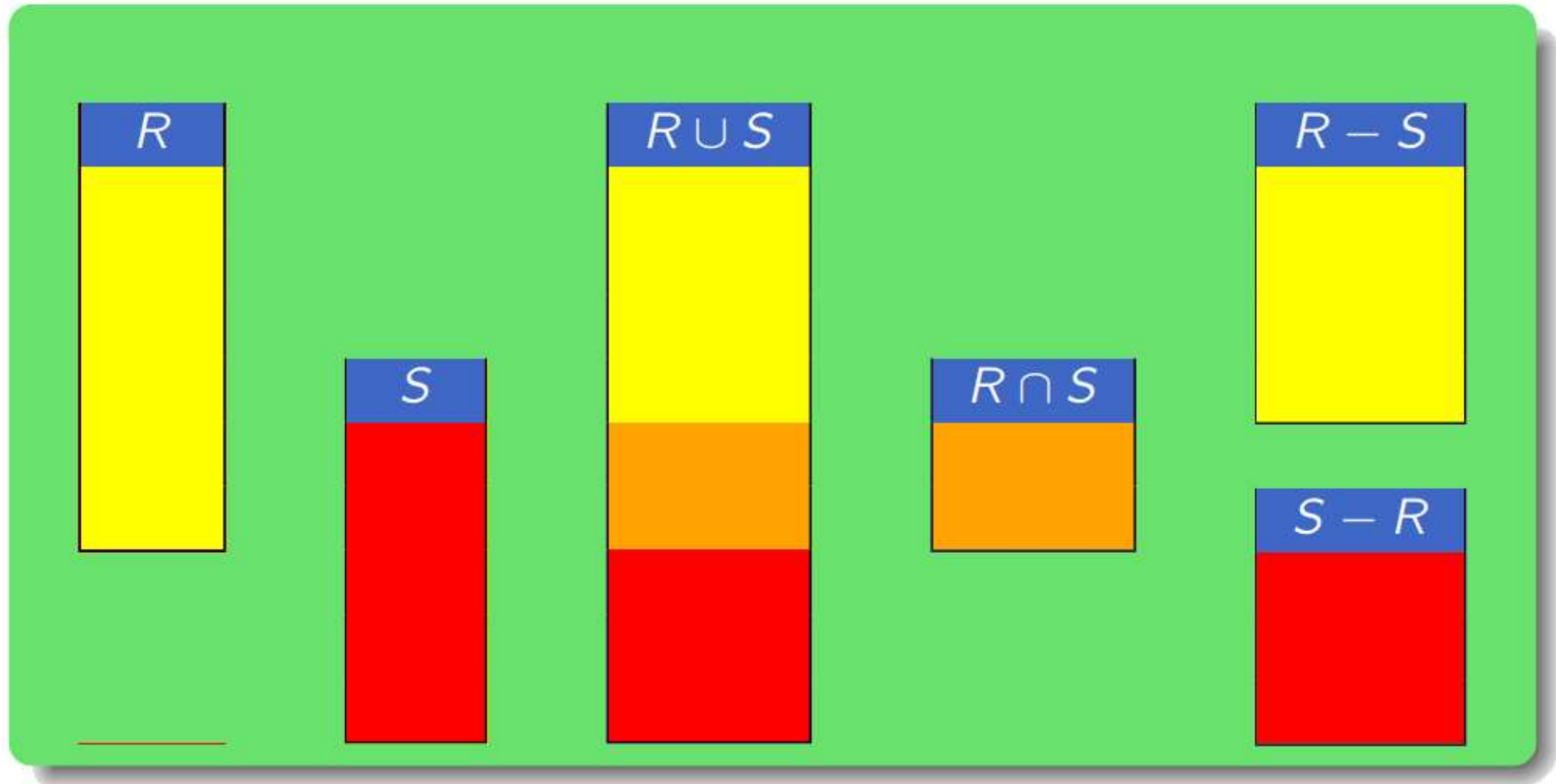
Set Operations (1)

- Relations are sets (of tuples). The “usual” family of binary set operations can also be applied to relations.
- It is a requirement, that both input relations have the same schema.

Set Operations

The set operations of relational algebra are $R \cup S$, $R \cap S$, and $R - S$ (union, intersection, difference).

Set Operations (2)



Union (1)

- In RA queries, a typical application for $\cup$ is case analysis.

Example: Grading

$\text{MPOINTS} := \pi_{\text{SID}, \text{POINTS}}(\sigma_{\text{CAT}='M' \wedge \text{ENO}=1}(\text{RESULTS}))$

$\pi_{\text{SID}, \text{GRADE} \leftarrow 'A'}(\sigma_{\text{POINTS} \geq 12}(\text{MPOINTS}))$
 $\cup \pi_{\text{SID}, \text{GRADE} \leftarrow 'B'}(\sigma_{\text{POINTS} \geq 10 \wedge \text{POINTS} < 12}(\text{MPOINTS}))$
 $\cup \pi_{\text{SID}, \text{GRADE} \leftarrow 'C'}(\sigma_{\text{POINTS} \geq 7 \wedge \text{POINTS} < 10}(\text{MPOINTS}))$
 $\cup \pi_{\text{SID}, \text{GRADE} \leftarrow 'F'}(\sigma_{\text{POINTS} \leq 7}(\text{MPOINTS}))$

Union (2)

- In SQL, $\cup$ is directly supported: keyword UNION.
UNION may be placed between two SELECT-FROM-WHERE blocks

SQL's UNION

```
SELECT SID, 'A' AS GRADE
FROM RESULTS
WHERE CAT = 'M' AND ENO = '1' AND POINTS >= 12
UNION
SELECT SID, 'B' AS GRADE
FROM RESULTS
WHERE CAT = 'M' AND ENO = '1'
AND POINTS >= 10 AND POINTS < 12
UNION
...
```

Set Difference (1)

- Note: the RA operators $\sigma, \pi, \times, \Join, \cup$ are monotonic by definition, *e.g.*:

$$R \subseteq S \quad \Rightarrow \quad \sigma_{\varphi}(R) \subseteq \sigma_{\varphi}(S) \quad .$$

- Then it follows that every query Q that exclusively uses the above operators behaves monotonically:
 - Let I_1 be a database state, and let $I_2 = I_1 \cup \{t\}$ (database state after insertion of tuple t).
 - Then every tuple u contained in the answer to Q in state I_1 is also contained in the answer to Q in state I_2 .

Database insertion never invalidates a correct answer.

Example Database (recap)

STUDENTS

<u>SID</u>	FIRST	LAST	EMAIL
101	Ann	Smith	...
102	Michael	Jones	(null)
103	Richard	Turner	...
104	Maria	Brown	...

EXERCISES

<u>CAT</u>	<u>ENO</u>	TOPIC	MAXPT
H	1	Rel.Alg.	10
H	2	SQL	10
M	1	SQL	14

RESULTS

<u>SID</u>	<u>CAT</u>	<u>ENO</u>	POINTS
101	H	1	10
101	H	2	8
101	M	1	12
102	H	1	9
102	H	2	9
102	M	1	10
103	H	1	5
103	M	1	7

Set Difference (2)

A correct solution?

$$\pi_{\text{FIRST, LAST}}(\text{STUDENTS} \bowtie_{\text{SID} \neq \text{SID2}} \pi_{\text{SID2} \leftarrow \text{SID}}(\text{RESULTS}))$$

A correct solution?

$$\pi_{\text{SID, FIRST, LAST}}(\text{STUDENTS} - \pi_{\text{SID}}(\text{RESULTS}))$$

Correct solution!

Set Operations and Complex Selections

- Note that the availability of $\cup$, $-$ (and $\cap$) renders complex selection predicates superfluous:

Predicate Simplification Rules

$$\sigma_{\varphi_1 \wedge \varphi_2}(Q) \xrightarrow{\Rightarrow} \sigma_{\varphi_1}(Q) \cap \sigma_{\varphi_2}(Q)$$

$$\sigma_{\varphi_1 \vee \varphi_2}(Q) = \sigma_{\varphi_1}(Q) \cup \sigma_{\varphi_2}(Q)$$

$$\sigma_{\neg \varphi}(Q) = Q - \sigma_{\varphi}(Q)$$

The five basic operations of relational algebra are:

- ① σ_φ **Selection**
- ② π_L **Projection**
- ③ $\times$ **Cartesian Product**
- ④ $\cup$ **Union**
- ⑤ $-$ **Difference**

- **Derived** (and thus redundant) **operations**:
Theta-Join $\bowtie_\theta$, **Natural Join** $\bowtie$, **Semi-Join** $\ltimes$, **Renaming** ρ
and **Intersection** $\cap$.